K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

DATA STRUCTURES

ASSIGNMENT 1

TOPIC : FOUNDATIONS AND ALGORITHMIC ANALYSIS

SUMBITTED TO :

MRS. NEETU CHAUHAN

SUMBITTED BY:

NAME : SANIA KAMBOJ

PROGRAM : B.TECH CSE (AI & ML)

ROLL NO : 2501730319

SECTION : A

SEMESTER : 2

BATCH : 2025-2029

Part A: Stack and ADTs

- **What is an ADT?**
- An Abstract Data Type (ADT) is a logical description of a data type that specifies **what** operations can be performed (like push/pop) but not **how** they are implemented.
- **O(1) Complexity Operations:**
- Stack operations like push and pop are O(1) because they only involve adding or removing an element from the top of the stack, requiring no loops or shifting of other elements.

Part B: Factorial & Fibonacci Analysis

Algorithm	**Time Complexity**	**Space Complexity**	**Paradigm**
Factorial	O(n)	O(n) (Stack depth)	Recursion
Fib_Naive	O(2^n)	O(n)	Recursion
Fib_Memo	O(n)	O(n)	Dynamic Programming

- **Why Naive Fibonacci is slow:**
- It performs redundant calculations. For example, to find fib(5), it calculates fib(3) multiple times across different branches of the recursion tree.

Part C: Tower of Hanoi

- **Moves (2^n - 1):** Each disk must be moved, and for each disk, the stack above it must be moved twice (once to the auxiliary peg and once to the destination). This doubling effect leads to the formula 2^n - 1.
- **Complexity:** Time Complexity is **O(2^n)** due to the exponential growth of moves; Space Complexity is **O(n)** representing the maximum depth of the recursion stack.

Part D: Binary Search

- **Recurrence:** $T(n) = T(n/2) + O(1) \Longrightarrow O(\log n)$

This specific formula describes **Recursive Binary Search**:

- **T(n)**: This represents the total time taken to solve a problem of size n.
- **T(n/2)**: Because Binary Search is a **Divide & Conquer** algorithm, it splits the list in half and only searches one of those halves. Therefore, the next step only has to deal with half the data (n/2).
- **O(1)**: This represents the "constant" amount of work done at each step—specifically, calculating the middle index and comparing the middle element to your target.

- **Cases: Best Case** is O(1) (key is the middle element). **Worst/Average Case** is O(log n) (key is at the ends or not present).

Part E: Paradigm Reflection

- **Divide & Conquer:** Binary Search splits the problem in half at every step.

  Real-life example: Searching for a word in a physical dictionary by opening it in the middle.

- **Dynamic Programming:** Memoized Fibonacci solves the "overlapping subproblems" issue by storing previous results.

  Real-life example: GPS navigation systems caching shortest paths between major intersections.

- **Greedy:** Makes the best local choice at each step.

  Real-life example: A cashier giving change using the fewest coins possible by always picking the largest denomination first.